

Trilha

Fórum

video	05:34
Projeto 2 - Expandindo Arrays Aninhados	05:22
Projeto 2 - Loops e Repetições	09:15
Projeto 2 - Pivot e Unpivot	06:14
Projeto 2 - Split Functions	07:14
Projeto 2 - Método Collect no Spark	01:39
Projeto 2 - Operadores	00:28
Projeto 2 - WithColumn	01:20
Projeto 2 - Operações de Colunas	01:32
Projeto 2 - MapType	02:01



MapType

No Apache Spark, **MapType** é um tipo de dado que representa uma coleção de pares chave-valor, semelhante a um dicionário em Python ou um objeto em JavaScript. É útil para armazenar e operar sobre dados que naturalmente se encaixam em um formato de mapa (ou dicionário).

O **MapType** pode ser empregado tanto em RDDs quanto em DataFrames, mas é mais comumente usado e acessível através da API DataFrame.

Definição de MapType

MapType é definido especificando os tipos de dados para as chaves e valores. Em PySpark, você pode definir um **MapType** usando **MapType(keyType, valueType)**, onde **keyType** e **valueType** podem ser qualquer tipo de dados Spark como

StringType, **IntegerType**, **ArrayType**, etc.

Exemplo de Criação e Uso de MapType

Vamos considerar um exemplo prático de como criar um DataFrame com uma coluna do tipo **MapType** e realizar operações básicas:

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.types import MapType, StringType, IntegerType
3 from pyspark.sql.functions import col
4
5 # Inicializa o SparkSession
6 spark = SparkSession.builder \
7     .appName("MapType Example") \
8     .getOrCreate()
9
10 # Dados de exemplo
11 data = [{"1", {"key1": 100, "key2": 200}}, {"2", {"key1": 300, "key2": 400}}]
12
13 # Definindo o esquema, incluindo uma coluna MapType
14 schema = ["id", "atributos"]
15
16 # Criando um DataFrame
17 df = spark.createDataFrame(data, schema)
18
19 # Mostrando o DataFrame
20 df.show(truncate=False)
21
22 # Acessando valores específicos do MapType
23 df.select(df["id"], df["atributos"]["key1"]).show()
24
25 # Adicionando uma nova chave-valor ao mapa
26 from pyspark.sql.functions import map_concat, lit
27
28 df = df.withColumn("novos_atributos", map_concat(df["atributos"], lit({"key3": 500})))
29 df.show(truncate=False)
30
```

Operações Comuns com MapType

- Acesso a Elementos:** Você pode acessar elementos específicos em um MapType usando a sintaxe `col("mapColumn")["key"]`. Isso retorna o valor associado com a chave especificada.
- Adicionar ou Atualizar Elementos:** Para adicionar ou atualizar elementos em um mapa, você pode usar funções como `map_concat` para mesclar mapas existentes com novos pares chave-valor.
- Remover Elementos:** A remoção de elementos específicos de um mapa pode ser feita criando uma UDF (User Defined Function) que manipula o dicionário como desejar, uma vez que operações de remoção direta não são diretamente suportadas.

Considerações sobre MapType

- Tipo de Chaves e Valores:** No Spark, as chaves em **MapType** são sempre tratadas como sendo de um tipo uniforme, assim como os valores. Diferentes tipos de chaves ou valores dentro do mesmo mapa não são permitidos.
- Desempenho:** Operar sobre **MapType** pode ser menos eficiente do que operar sobre colunas individuais, especialmente se o mapa contiver muitos elementos ou se operações complexas forem aplicadas frequentemente.
- Uso:** **MapType** é particularmente útil em casos de uso como agregação de múltiplos atributos em uma única coluna ou quando trabalhando com dados semi-estruturados.

✔ MapType é uma ferramenta poderosa dentro do ecossistema Spark, oferecendo flexibilidade na manipulação de dados complexos. Contudo, é importante considerar as implicações de desempenho ao projetar soluções que dependem intensamente de mapas.



Equipe DSA

Continue trilhando uma excelente
jornada de aprendizagem.

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte

🔍 Suporte